

Dataclenz Library: Search Algorithm Implementation and Performance Analysis

Dataclenz Development Team

May 7, 2025

Contents

1	Introduction	2
2	Overview of Dataclenz Library	2
3	Search Algorithm Implementations	2
3.1	Binary Search Implementation	2
3.1.1	Algorithm Description	2
3.1.2	Time and Space Complexity	2
3.1.3	Implementation in Dataclenz	3
3.2	Jump Search Implementation	4
3.2.1	Algorithm Description	4
3.2.2	Time and Space Complexity	4
3.2.3	Implementation in Dataclenz	4
4	Performance Analysis	5
4.1	Methodology	5
4.2	Results	6
4.2.1	Average Search Times	6
4.2.2	Performance by Dataset Size	6
4.2.3	Impact of Data Type	6
4.3	Real-World Performance	6
5	Optimizations and Implementation Details	7
5.1	Data Type Handling	7
5.2	Memory Management	7
5.3	Multiple Match Handling	7
6	Recommendations and Best Practices	7
7	Conclusion	8

1 Introduction

This document provides a comprehensive overview of the search functionality implemented in the Dataclenz library. Dataclenz is a C-based data manipulation library designed for efficient handling, analysis, and transformation of structured data. This document focuses specifically on the search algorithms implemented in the library, their theoretical foundations, implementation details, and performance characteristics.

The Dataclenz library implements several search algorithms with a focus on binary search and jump search, which are optimized for various data types and dataset sizes. These algorithms are critical components of data analysis operations, enabling fast data retrieval, filtering, and querying capabilities.

2 Overview of Dataclenz Library

Dataclenz is a comprehensive data manipulation library written in C. It provides a flexible DataFrame-like structure that supports:

- Multiple data types (integers, floats, strings)
- Data import/export from CSV files
- Data filtering and transformation
- Statistical analysis and descriptive statistics
- Efficient searching and sorting algorithms
- Memory-efficient data structures

The library is designed to be lightweight yet powerful, making it suitable for applications ranging from embedded systems to desktop applications requiring efficient data processing capabilities.

3 Search Algorithm Implementations

3.1 Binary Search Implementation

Binary search is a divide-and-conquer algorithm that finds the position of a target value within a sorted array. The algorithm compares the target value to the middle element of the array; if they are unequal, the half in which the target cannot lie is eliminated and the search continues on the remaining half until the target value is found or the subarray is empty.

3.1.1 Algorithm Description

Binary search operates by repeatedly dividing the search interval in half. If the value of the search key is less than the item in the middle of the interval, the interval is narrowed to the lower half. Otherwise, it is narrowed to the upper half. The algorithm terminates when the item is found or the interval is empty.

3.1.2 Time and Space Complexity

- Time Complexity: $O(\log n)$
- Space Complexity: $O(1)$

3.1.3 Implementation in Dataclenz

Below is a simplified representation of the binary search implementation in the Dataclenz library:

```
1 int binary_search(DataFrame* df, int column_index, void* target,
2                   int* found_indices, int max_indices) {
3     if (df == NULL || column_index < 0 || column_index >= df->
4         num_columns
5         || found_indices == NULL) {
6         set_error("Invalid parameters for binary search");
7         return 0;
8     }
9     Column* column = &df->columns[column_index];
10    int found_count = 0;
11
12    if (df->num_rows == 0) {
13        return 0;
14    }
15
16    int left = 0;
17    int right = df->num_rows - 1;
18    int mid = 0;
19    int compare_result = 0;
20
21    // Binary search to find any occurrence
22    while (left <= right) {
23        mid = left + (right - left) / 2;
24
25        // Compare based on data type
26        switch (column->type) {
27            case TYPE_INT:
28                compare_result = *((int*)target - ((int*)column->data)[
29                    mid];
30                break;
31            case TYPE_FLOAT:
32                {
33                    float diff = *((float*)target - ((float*)column->
34                        data)[mid];
35                    compare_result = (diff > 0) ? 1 : ((diff < 0) ? -1
36                        : 0);
37                }
38                break;
39            case TYPE_STRING:
40                compare_result = strcmp((char*)target, ((char**)column
41                    ->data)[mid]);
42                break;
43        }
44
45        if (compare_result == 0) {
46            // Found a match - store index and check for more
47            found_indices[found_count++] = mid;
48
49            // Check for more matches to the left and right
50            // [code omitted for brevity]
51
52            return found_count;
53        } else if (compare_result < 0) {
54            right = mid - 1;
55        }
56    }
57}
```

```

51         } else {
52             left = mid + 1;
53         }
54     }
55
56     return found_count;
57 }

```

Listing 1: Binary Search Implementation in Dataclenz

The implementation handles multiple data types (integer, float, string) and returns all matching indices, not just the first occurrence.

3.2 Jump Search Implementation

Jump search is a searching algorithm for sorted arrays that works by jumping ahead by fixed steps and then performing a linear search within a smaller range where the element might be present.

3.2.1 Algorithm Description

Jump search works by jumping ahead by fixed steps of size $\sqrt{n}$ where n is the length of the array. Once we find an interval where the target value may exist (when the value at the current position is greater than the target), we perform a linear search in that interval.

3.2.2 Time and Space Complexity

- Time Complexity: $O(\sqrt{n})$
- Space Complexity: $O(1)$

3.2.3 Implementation in Dataclenz

Below is a simplified representation of the jump search implementation:

```

1  int jump_search(DataFrame* df, int column_index, void* target,
2                  int* found_indices, int max_indices) {
3      if (df == NULL || column_index < 0 || column_index >= df->
num_columns
4          || found_indices == NULL) {
5          set_error("Invalid parameters for jump search");
6          return 0;
7      }
8
9      Column* column = &df->columns[column_index];
10     int found_count = 0;
11     int n = df->num_rows;
12
13     if (n == 0) {
14         return 0;
15     }
16
17     // Finding block size to be jumped
18     int step = (int)sqrt(n);
19
20     // Finding the block where element is present
21     int prev = 0;
22     int compare_result = 0;

```

```

23
24     while (prev < n) {
25         // Compare based on data type
26         switch (column->type) {
27             case TYPE_INT:
28                 compare_result = *((int*)target - ((int*)column->data)[
prev];
29                 break;
30             case TYPE_FLOAT:
31                 {
32                     float diff = *((float*)target - ((float*)column->
data)[prev];
33                     compare_result = (diff > 0) ? 1 : ((diff < 0) ? -1
: 0);
34                 }
35                 break;
36             case TYPE_STRING:
37                 compare_result = strcmp((char*)target, ((char**)column
->data)[prev]);
38                 break;
39         }
40
41         if (compare_result == 0) {
42             // Found a match
43             found_indices[found_count++] = prev;
44
45             // Check for more matches in the vicinity
46             // [code omitted for brevity]
47
48             return found_count;
49         } else if (compare_result < 0) {
50             // Element is in this block, perform linear search
51             break;
52         }
53
54         prev = (prev + step < n) ? prev + step : n;
55     }
56
57     // Linear search in the identified block
58     // [code omitted for brevity]
59
60     return found_count;
61 }

```

Listing 2: Jump Search Implementation in Dataclenz

Like the binary search implementation, jump search handles multiple data types and returns all matching indices.

4 Performance Analysis

We conducted extensive performance testing of the search algorithms implemented in Dataclenz, comparing binary search and jump search across various dataset sizes and data types. This section presents the findings from those tests.

4.1 Methodology

The performance analysis methodology involved:

- Testing on synthetic datasets ranging from 50 to 100,000 elements
- Testing on real-world datasets (MoMA artists and housing data)
- Conducting 1,000 searches for each combination of algorithm, dataset size, and data type
- Measuring average search time, number of comparisons, and memory usage
- Testing across three data types: integers, floats, and strings

All datasets were sorted before searching, as both binary search and jump search require sorted data to function correctly.

4.2 Results

4.2.1 Average Search Times

Our testing revealed that binary search consistently outperformed jump search, particularly as dataset sizes increased:

Table 1: Average Search Times by Algorithm

Algorithm	Average Time (seconds)
Binary Search	0.00000016
Jump Search	0.00000042

4.2.2 Performance by Dataset Size

The performance gap between binary search and jump search widened significantly as dataset size increased. For the largest dataset (100,000 elements):

Table 2: Performance at Largest Dataset Size (100,000 elements)

Algorithm	Search Time (seconds)	Relative Performance
Binary Search	0.00000033	1x
Jump Search	0.00000133	4x slower

This confirms the theoretical expectation that binary search ($O(\log n)$) scales better than jump search ($O(\sqrt{n})$) for large datasets.

4.2.3 Impact of Data Type

Testing across different data types revealed that:

- String comparisons were generally slower than numeric comparisons
- The relative performance difference between algorithms remained consistent across data types
- For all data types, binary search demonstrated better scaling with increased dataset size

4.3 Real-World Performance

Tests on real-world datasets confirmed the findings from synthetic data. When searching through the MoMA artists dataset and housing dataset, binary search consistently outperformed jump search, particularly for larger datasets and string-based searches.

5 Optimizations and Implementation Details

The search implementations in Dataclenz include several key optimizations:

5.1 Data Type Handling

Both search algorithms are implemented to handle various data types transparently:

- Integers: Direct comparison of values
- Floats: Comparison with handling for floating-point precision issues
- Strings: Lexicographical comparison using strcmp

5.2 Memory Management

The implementations are designed to be memory-efficient:

- Both algorithms have $O(1)$ space complexity
- Memory for search results is pre-allocated based on the maximum possible number of matches
- Temporary memory is properly freed to prevent memory leaks

5.3 Multiple Match Handling

Unlike simple search implementations that only return the first occurrence, the Dataclenz search algorithms:

- Return all occurrences of the target value
- Efficiently scan adjacent elements when a match is found
- Support a configurable maximum number of matches to return

6 Recommendations and Best Practices

Based on our analysis, we recommend the following practices when using the search functionality in Dataclenz:

- **Use Binary Search for Large Datasets:** Binary search demonstrates superior performance for datasets larger than 1,000 elements.
- **Pre-sort Data:** Both search algorithms require sorted data. Sort once and search multiple times when possible.
- **Consider Jump Search for Medium Datasets:** For datasets between 100-1,000 elements, the performance difference is less significant, and jump search may be adequate.
- **String Searches:** For string-heavy datasets, the performance advantage of binary search is even more pronounced.

7 Conclusion

The search functionality in the Dataclenz library provides efficient and versatile data retrieval capabilities. By implementing both binary search and jump search algorithms with support for various data types, the library offers flexible solutions for different use cases and dataset characteristics.

Our performance analysis confirms the theoretical expectation that binary search outperforms jump search for larger datasets, while both algorithms provide adequate performance for smaller datasets. The implementations are optimized for memory efficiency and handling of multiple matches, making them suitable for real-world data processing applications.

The Dataclenz library continues to evolve with ongoing performance optimizations and feature enhancements to meet the needs of data-intensive applications across various domains.